

Day 42:-

* Functions in Python :-

Index:

=> purpose of functions.

=> Types of languages.

① un-structured programming languages.

② structured programming languages.

=> Definition of function.

=> parts of function.

=> phases in function.

=> syntax for defining functions.

=> programming examples.

=

=> Arguments and parameters.

=> Types of Arguments

① positional Arguments.

② default Arguments.

③ keyword Arguments.

④ variable length Arguments.

⑤ keyword variable length Arguments.

=> programming examples.

=> Global and local variables

=> programming examples.

=> global keywords

=> global()

=> programming example.

=> Anonymous or lambda function.

=> Implementation of Anonymous or lambda function.

=> Special Function in Python :-

① filter() with Normal and Anonymous Function.

② map()

③ reduce()

=

* Types of Languages:-

=> In the context of Functions, we have two types of programming languages. They are:

- (a) unstructured programming lang.
- (b) structured programming lang.

(a) unstructured programming lang:-

=> In un-structured programming language, we don't have the concept of Functions.

=> Since un-structured programming language does not contain the concept of functions and hence, whose application having the following limitations.

- ① Application development is more.
- ② Application takes more memory space.
- ③ Application execution time is more.
- ④ Application performance is degraded.
- ⑤ ~~Application~~ Redundancy (Duplication or replication) of the code is more.

examples: GW-BASIC

(b) structured programming lang:-

=> In structured programming lang, we have the concept of Functions.

=> Since structured programming language contains the concept of functions and hence, whose applications having the following advantages.

- ① Application development time is less.
- ② Application takes less memory space.
- ③ Application execution time is less.
- ④ Application performance is enhanced (improved).
- ⑤ Redundancy (Duplication or replication) of the code is minimized.

eg) C, C++, Java, Python etc.

ATM Project/Application.

- => Deposit()
- => Withdraw()
- => Bal Enq()
- => pin generate()
- => pin Change()
- => Fast Cash()
- => mobile number reg()

⇒ sub program of main program is called "Functions"

(OR)

$\Rightarrow$ A part of main program is called function

maths - function

consider, ^① def ^② $f(x, y) = x + y$ ^④ - - - - - Function definition

Find ① $f(2,3)$ ② $f(-4,-5)$ $\leftarrow$ Function calls.

Function Heading

```
def f(x,y):
```

$z = x + y$
 return z

function body

Function Def = Function Heading + Function Body

(1) def (2) (3)
f(x, y):

(4) " " " " This function computes sum of two numbers " " " "

```
(6) z = x + y(5)
    return z.
```

* Functions in Python :-

→ The purpose of functions is that "To perform certain operation / task and provides code re-usability".

→ The Advantages of Functions in any language are:-

- ① Application development ^{time} is less.
- ② " memory space is less.
- ③ " Execution time is less.
- ④ " performance is enhanced.
- ⑤ Redundancy of the code is minimized.

* Definitions of function

- sub program of main program is called Function.
(OR)
- A part of main program is called function.

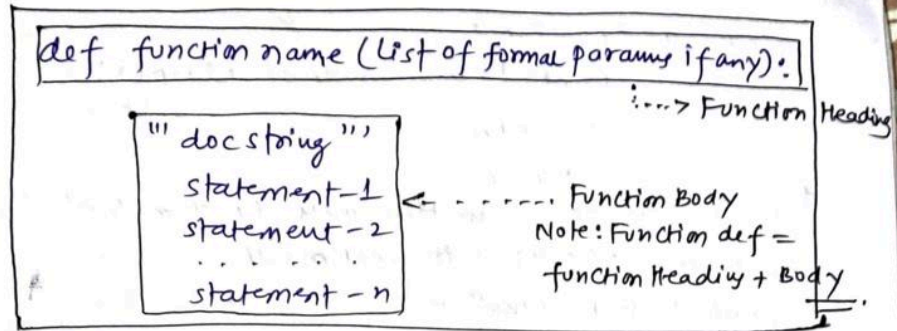
* Parts of function:-

- At the time developing functions in real time, we must ensure that, there must exist 2 parts. they are:
 - ① function definition
 - ② function calls.
- Every function Definition exists only once.
- Every function call must contains a function Definition otherwise we get Nameerror.
- Function Definition will be execute when we call by using functions calls OR without calling the Function.
by using Function calls pvm will not execute function definition.

* Phases in Functions:-

- At the time Defining the functions, the programmer must ensure that there must exist the following phases.
 - ① Every Function must take input
 - ② Every Function must process the input
 - ③ Every Function must give output or result

* syntax for defining functions:-



- ① Here 'def' is a keyword, which is used for defining programmer-defined functions.
- ② 'function name' represents a valid variable name and treated as function name and every function name is an object of type <class, 'function'>
- ③ 'List of formal params' represent list of variable names used in function heading and they are used for storing or holding the input(s) coming from function call(s).
- ④ '"doc string"' represent documentation string and it is used for giving or writing the description about functionality of function.
- ⑤ The statement 1, statement 2 ... stmt n indentation block of statement and it is process the input or logic for problem solving and it is known Business logic.
- ⑥ In the function body, we use some special variables and they are called local variables and whose purpose is to store the temporary results.
- ⑦ The values of Formal Params and Local Variables can be accessed only inside corresponding function definition but not possible to access in other part of the program and in other function definitions (scope of formal params & local var)

this prog. computes sum of two numbers by using functions:-

```
def addop(a,b):
    print("i am inside of addop()")
    c = a+b
    return c
# print("Line 8: I am here after function defn:")
res = addop(3,4) # function call
print("Line 10 - sum", res)
res = addop(100,200) # function call
print("sum", res)
```

pehle
I direct print kar diya
kyunki usko function
call nhi mila hai

program computes sum of two numbers using functions

```
In [5]: def sumop(x,y):
        print("i am inside a loop")
        z=x+y
        return z
        print("i am the after function defination")
        res=sumop(4,5)
        print("sum:",res)
        res1=sumop(18,18)
        print("sum:",res1)
```

```
i am the after function defination
i am inside a loop
sum: 9
i am inside a loop
sum: 36
```

In []: